Computer Organization and Architecture Laboratory

MiniRISC Assignment 2A: Data BRAM and Load/Store Integration

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Deadline: 12 October 2026

Objective

Extend the MiniRISC processor developed up to 7 October by integrating a **BRAM-based data memory** and implementing complete load/store functionality.

By the end of this assignment, the processor should be able to:

- compute a data-memory effective address;
- read a 32-bit word from data BRAM using `LD`;
- write a 32-bit word to data BRAM using `ST`;
- select between an ALU result and memory data for register write-back;
- execute a short deterministic memory-access sequence in behavioral simulation.

The multiplier/MAC unit is **not** part of this assignment.

1. Memory Addressing Convention

Use a **32-bit word-addressed data memory**. Each data-memory address identifies one complete 32-bit word.

For example,

```
LD R4, 3(R5)
```

means

$$R4 \leftarrow Mem[R5 + 3].$$

Similarly,

```
ST R6, 2(R7)
```

means

$$Mem[R7 + 2] \leftarrow R6.$$

No 8-bit or 16-bit load/store instruction is required.

2. Mandatory Data BRAM

The data memory must be implemented using FPGA **Block RAM (BRAM)**. A Verilog register array must not be used as the processor data memory.

The BRAM may be configured as:

- **Single-Port RAM**, or
- **Dual-Port RAM**,

depending on the group's processor organization.

For the baseline MiniRISC processor, a single-port RAM is sufficient if only one data-memory operation is performed at a time.

A suitable common configuration is:

- word width: 32 bits;
- depth: chosen by the group, but at least 256 words is recommended;
- synchronous read;
- write enable;
- initialization through a `.coe` file for deterministic testing.

A conceptual single-port RAM interface is:

```
data_bram data_mem (
    .clka  (clk),
    .ena   (mem_en),
    .wea   (mem_we),
    .addra (mem_addr),
    .dina  (mem_write_data),
    .douta (mem_read_data)
);
```

The exact generated BRAM IP port names may differ.

3. Effective Address Generation

Both `LD` and `ST` should support a base-register plus immediate-offset address:

$$EA = R_{base} + \mathrm{Ext}(offset).$$

Example:

```
LD R4, 5(R2)
```

requires

$$EA = R2 + 5.$$

Similarly,

`ST R7, -2(R3)`

requires

$$EA = R3 - 2.$$

The existing ALU may be reused for this addition. A separate adder is not mandatory.

4. Required LD Datapath

For

`LD R4, 5(R2)`

the processor should conceptually perform:

$$R2 + 5 \rightarrow EA \rightarrow Data\ BRAM \rightarrow Memory\ Read\ Data \rightarrow Write\ Back \rightarrow R4.$$

Required functionality:

1. read the base register;
2. extend the offset;
3. calculate the effective address;
4. apply the address to data BRAM;
5. wait for valid synchronous BRAM output;
6. select memory data at the write-back multiplexer;
7. write the loaded word into the destination register.

5. Required ST Datapath

For

`ST R6, 2(R5)`

the processor should compute

$$EA = R5 + 2$$

and then perform

$$Mem[EA] \leftarrow R6.$$

No register write-back should occur for `ST`.

6. Write-Back Selection

The register-file write-data path must now support at least:

$$WriteData = \begin{cases} ALUResult, & \text{arithmetic/logical instruction} \\ MemReadData, & \texttt{LD} \end{cases}$$

Include a write-back selection signal, e.g. `wb_sel`, or an equivalent group-defined signal.

7. Minimum Control Signals

Equivalent control must exist for at least:

Control	**Purpose**
ALU operation select	Select address addition or normal ALU operation
ALU source select	Select register or immediate/offset
Memory enable	Enable data BRAM access
Memory write enable	Distinguish RAM write from read
Register write enable	Enable destination-register update
Write-back select	Select ALU result or memory read data
Destination-register select	Select the correct destination field

The complete final processor FSM is not yet required, but the control sequence for `LD` and `ST` must be correct.

8. Synchronous BRAM Timing

Students must account for synchronous BRAM timing. For a typical one-cycle read latency:

$$\text{Cycle } n: \ EA \text{ is applied to BRAM}$$

and

$$\text{Cycle } n+1: \ MemReadData \text{ becomes available.}$$

The control logic must not write an old or invalid BRAM output into the register file.

9. Required Test Data

Initialize part of the data BRAM using a `.coe` file.

A suggested memory image is:

Word Address	Initial 32-bit Value
0	10
1	20
2	30
3	40
4	50
5	60
6	70
7	80

10. Minimum Behavioral Demonstration

Case 1: Load. Assume $R2 = 2$. Execute an instruction equivalent to:

```
LD R4, 3(R2)
```

Then

$$EA = 2 + 3 = 5, \qquad R4 = Mem[5] = 60.$$

Case 2: Store. Assume $R5 = 1$ and $R6 = 99$. Execute:

```
ST R6, 4(R5)
```

Then

$$EA = 1 + 4 = 5, \qquad Mem[5] = 99.$$

Case 3: Read back. Execute:

```
LD R7, 4(R5)
```

and verify

$$R7 = 99.$$

11. Signals to Show in the Waveform

Show at least:

- clock and reset;
- PC and fetched instruction;
- decoded base/source/destination fields;
- base-register value and extended offset;
- ALU/effective-address result;
- data-BRAM address, write data, read data, and write enable;
- register write enable and write-back select;
- destination-register value after `LD`.

12. Suggested Module Organization

```
minirisc32_top
|
|-- pc_unit
|-- instruction_bram_rom
|-- instruction_decoder
|-- register_file_16x32
|-- immediate_extension_logic
|-- alu_input_mux
|-- alu32
|-- data_bram_ram
|-- memory_control
|-- writeback_mux
`-- baseline_control_logic
```

A separate `effective_address_unit` is optional if the existing ALU performs address addition.

13. Explicitly Not Required on 12 October

The following are deferred:

- complete branch control;
- final baseline control FSM;
- Booth multiplier integration;
- `MUL`/`MAC` execution;
- multiplier stall/`NOP` scheduling;
- matrix multiplication;
- convolution;
- UART output.

14. Submission and Demonstration

Each group should submit/show:

- updated Verilog source files;
- data-BRAM IP configuration;
- data-memory `.coe` file;
- behavioral testbench and simulation waveform;
- updated architecture diagram showing the data-memory path;
- a short note stating whether single-port or dual-port RAM was used and why.